

Documentación de creación de usuarios

1) Creación de Usuarios

Los cambios que fueron demostrados mediante capturas no son guardados por flyway, porque flyway solamente guarda los cambios realizados mediante scripts.

The image displays two screenshots of the 'Inicio de sesión - Nuevo' (New Login) dialog box in SQL Server Enterprise Manager, showing the configuration for a new user.

Left Screenshot (lab_reader):

- Nombre de inicio de sesión:** lab_reader
- Autenticación:** Autenticación de SQL (selected)
- Contraseña:** [Redacted]
- Confirmar contraseña:** [Redacted]
- Exigir directivas de contraseña:** [Checked]
- Exigir expiración de contraseña:** [Unchecked]
- Exigir el usuario debe cambiar la contraseña en el siguiente inicio de sesión:** [Unchecked]
- Asignado a certificado:** [Unchecked]
- Asignado a clave asimétrica:** [Unchecked]
- Asignar a credencial:** [Unchecked]
- Credenciales asignadas:** [Empty list]
- Base de datos predeterminada:** Gathel - Gaming the life
- Idioma predeterminado:** <predeterminado>

Right Screenshot (lab_auditor):

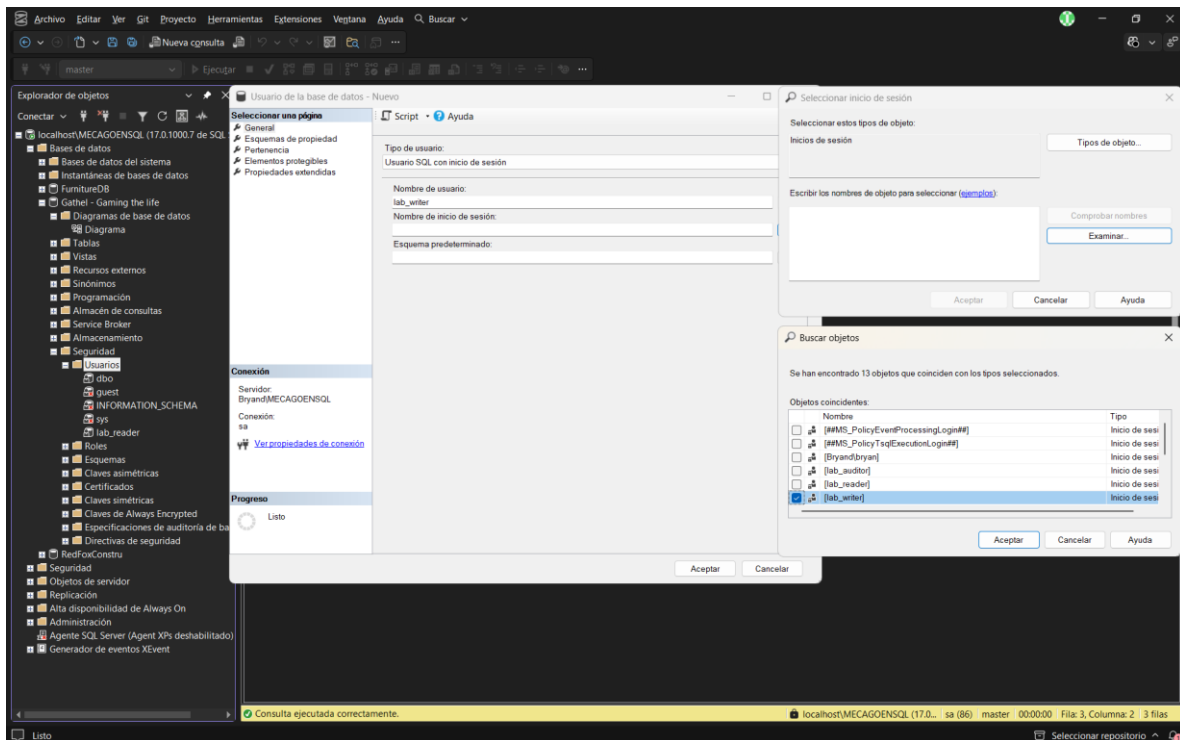
- Nombre de inicio de sesión:** lab_auditor
- Autenticación:** Autenticación de SQL (selected)
- Contraseña:** [Redacted]
- Confirmar contraseña:** [Redacted]
- Exigir directivas de contraseña:** [Checked]
- Exigir expiración de contraseña:** [Checked]
- Exigir el usuario debe cambiar la contraseña en el siguiente inicio de sesión:** [Checked]
- Asignado a certificado:** [Unchecked]
- Asignado a clave asimétrica:** [Unchecked]
- Asignar a credencial:** [Unchecked]
- Credenciales asignadas:** [Empty list]
- Base de datos predeterminada:** Gathel - Gaming the life
- Idioma predeterminado:** <predeterminado>

The image displays a screenshot of the 'Inicio de sesión - Nuevo' (New Login) dialog box in SQL Server Enterprise Manager, showing the configuration for a new user.

Left Screenshot (lab_writer):

- Nombre de inicio de sesión:** lab_writer
- Autenticación:** Autenticación de SQL (selected)
- Contraseña:** [Redacted]
- Confirmar contraseña:** [Redacted]
- Exigir directivas de contraseña:** [Checked]
- Exigir expiración de contraseña:** [Unchecked]
- Exigir el usuario debe cambiar la contraseña en el siguiente inicio de sesión:** [Unchecked]
- Asignado a certificado:** [Unchecked]
- Asignado a clave asimétrica:** [Unchecked]
- Asignar a credencial:** [Unchecked]
- Credenciales asignadas:** [Empty list]
- Base de datos predeterminada:** Gathel - Gaming the life
- Idioma predeterminado:** <predeterminado>

2) Creación del Usuario en la base de datos



Se realizó exactamente lo mismo para los otros 2 usuarios

3) Creación de roles

GO

```
CREATE ROLE rl_gathel_reader;
```

```
CREATE ROLE rl_gathel_writer;
```

```
CREATE ROLE rl_gathel_auditor;
```

GO

4) Asignación de roles a los usuarios

```
ALTER ROLE rl_gathel_reader ADD MEMBER lab_reader;
```

```
ALTER ROLE rl_gathel_writer ADD MEMBER lab_writer;
```

```
ALTER ROLE rl_gathel_auditor ADD MEMBER lab_auditor;
```

GO

5) Creación de permisos directos y heredados

- lab_reader tendrá SELECT directo sobre dbo.countries
- y heredará SELECT sobre dbo.propositions mediante el rol rl_gathel_reader

```
GRANT SELECT ON dbo.countries TO lab_reader;  
GRANT SELECT ON dbo.propositions TO rl_gathel_reader;  
GO
```

Un permiso directo y otro heredado. En SQL Server los permisos pueden asignarse directamente a usuarios o a roles, y luego heredarse por pertenencia al rol.

Un usuario sin SELECT, pero con acceso por Stored Procedure

La idea es:

- negar SELECT directo sobre dbo.players
- permitir ejecutar un procedimiento almacenado que sí devuelve información controlada

SQL Server permite que un procedimiento se ejecute con contexto de otro principal usando EXECUTE AS, incluido OWNER.

Crear el procedimiento

```
USE Gathel;  
GO
```

```
CREATE OR ALTER PROCEDURE dbo.sp_GetPlayerPublicProfile  
    @PlayerID BIGINT  
WITH EXECUTE AS OWNER  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    SELECT playerID, username, firstName, lastName, isActive  
    FROM dbo.players  
    WHERE playerID = @PlayerID;  
END;  
GO
```

Bloquea el SELECT directo

```
DENY SELECT ON dbo.players TO lab_reader;  
GO
```

Autoriza el procedimiento

```
GRANT EXECUTE ON dbo.sp_GetPlayerPublicProfile TO lab_reader;  
GO
```

6) Lectura sí / escritura no, y viceversa

Usuario lector sin escritura

```
GRANT SELECT ON dbo.propositions TO lab_reader;  
DENY INSERT, UPDATE, DELETE ON dbo.propositions TO lab_reader;  
GO
```

Usuario escritor sin lectura

```
GRANT INSERT ON dbo.loginAttempts TO lab_writer;  
DENY SELECT, UPDATE, DELETE ON dbo.loginAttempts TO lab_writer;  
GO
```

Entonces, cuando inicie sesión en lab reader voy a poder seleccionar datos de propositions pero no insertar, actualizar ni borrar, mientras que si inicio sesión en lab writer voy a poder insertar en loginAttempts pero no seleccionar, actualizar ni borrar.

7) Data Masking en datos sensibles

Se recomienda aplicar masking a:

players.email, players.firstName, players.lastName, loginAttempts.attemptedEmail,
loginAttempts.ipAddress, resources.contentURL,
paymentAttempts.transactionReference, paymentAttempts.requestPayload
paymentAttempts.responsePayload, playersSocialNetwork.externalAccountID
playersSocialNetwork.externalUsername,
playerSocialNetworkTokens.accessTokenHash,
playerSocialNetworkTokens.refreshTokenHash, auditLogs.previousValues
auditLogs.newValues, aiRequests.requestPayload y aiResponses.responsePayload

Pero solamente se aplicarán a tres para este ejemplo.

```
ALTER TABLE dbo.players  
ALTER COLUMN email ADD MASKED WITH (FUNCTION = 'email()');  
GO
```

```
ALTER TABLE dbo.loginAttempts
ALTER COLUMN attemptedEmail ADD MASKED WITH (FUNCTION = 'email()');
GO
```

```
ALTER TABLE dbo.loginAttempts
ALTER COLUMN ipAddress ADD MASKED WITH (FUNCTION =
'partial(0,"xxx.xxx.xxx.",4)');
GO
```

Nota: ninguno de los tres usuarios podrá ver correctamente los datos de estas tres tablas, a no ser que se les de permiso con el script GRANT UNMASK TO lab_auditor; por ejemplo, para autorizar al auditor a ver esos datos, no se aplicó aun a la base de datos para utilizarla como ejemplo en la revisión, otro punto es que si entramos como “sa” si podremos ver todos los datos masked.

8) Row-Level Security sobre propositions

Row-Level Security permite controlar el acceso por filas usando el contexto de ejecución o la membresía a grupos/roles. Microsoft indica que se implementa con una función inline y una SECURITY POLICY.

Primero se ejecuta

```
GO
```

```
CREATE SCHEMA SecurityLab;
GO
```

Segundo

```
CREATE TABLE SecurityLab.UserPlayerMap (
dbUserName SYSNAME NOT NULL PRIMARY KEY,
playerID BIGINT NOT NULL UNIQUE
);
GO
```

Tercero

Inserta mapeos de prueba

```
INSERT INTO SecurityLab.UserPlayerMap (dbUserName, playerID)
VALUES
('lab_reader', 1),
```

```
('lab_writer', 2),  
('lab_auditor', 3);  
GO
```

Crea la función de filtro

```
CREATE OR ALTER FUNCTION SecurityLab.fn_PropositionAccess  
(  
    @creatorPlayerID BIGINT,  
    @targetPlayerID BIGINT  
)  
RETURNS TABLE  
WITH SCHEMABINDING  
AS  
RETURN  
    SELECT 1 AS access_result  
    WHERE EXISTS (  
        SELECT 1  
        FROM SecurityLab.UserPlayerMap m  
        WHERE m.dbUserName = USER_NAME()  
        AND (m.playerID = @creatorPlayerID OR m.playerID = @targetPlayerID)  
    )  
    OR IS_MEMBER('rl_gathel_auditor') = 1;  
GO
```

¿Qué hace?

Supongamos que entra: lab_reader y SQL Server está leyendo esta fila:

propositionID	creatorPlayerID
1	1

Entonces pregunta:

¿USER_NAME() = lab_reader tiene playerID = 1?

Sí.

La fila pasa.

Ahora revisa esta:

propositionID	creatorPlayerID
2	2

Pregunta: ¿lab_reader tiene playerID = 2? No.

La fila “desaparece”.

Crea la política RLS

```
CREATE SECURITY POLICY SecurityLab.PropositionSecurityPolicy
ADD FILTER PREDICATE SecurityLab.fn_PropositionAccess(creatorPlayerID,
targetPlayerID)
ON dbo.propositions
WITH (STATE = ON);
GO
```

Esto hace que

```
SELECT *
FROM dbo.propositions;
```

automáticamente devuelve resultados distintos según quién esté conectado.

9) Cifrado con master key + certificate + symmetric key

Primero se crea la master key

GO

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Gathel#2026!MasterKey';
GO
```

Se crea el certificado

```
CREATE CERTIFICATE GathelPasswordCert
WITH SUBJECT = 'Certificate for Gathel sensitive secrets';
GO
```

Se crea la symmetric key

```
CREATE SYMMETRIC KEY GathelPasswordKey
WITH ALGORITHM = AES_256
```

```
ENCRYPTION BY CERTIFICATE GathelPasswordCert;  
GO
```

Se encripta una columna en una tabla de prueba

```
ALTER TABLE dbo.playerSocialNetworkTokens  
ADD encryptedAccessToken VARBINARY(MAX) NULL,  
encryptedRefreshToken VARBINARY(MAX) NULL;  
GO
```

Se cifran los datos

```
OPEN SYMMETRIC KEY GathelPasswordKey  
DECRYPTION BY CERTIFICATE GathelPasswordCert;  
GO
```

```
UPDATE dbo.playerSocialNetworkTokens  
SET encryptedAccessToken = ENCRYPTBYKEY(  
    KEY_GUID('GathelPasswordKey'),  
    CONVERT(NVARCHAR(500), accessTokenHash),  
    1,  
    CONVERT(VARBINARY(128), playerSocialNetworkTokenID)  
)  
encryptedRefreshToken = ENCRYPTBYKEY(  
    KEY_GUID('GathelPasswordKey'),  
    CONVERT(NVARCHAR(500), refreshTokenHash), 1,  
    CONVERT(VARBINARY(128), playerSocialNetworkTokenID)  
);  
GO  
CLOSE SYMMETRIC KEY GathelPasswordKey;  
GO
```


Y para descifrar los datos se utiliza el siguiente script

```
OPEN SYMMETRIC KEY GathelPasswordKey
DECRYPTION BY CERTIFICATE GathelPasswordCert;
GO
```

```
SELECT
playerSocialNetworkTokenID,
CONVERT(NVARCHAR(500), DECRYPTBYKEY(encryptedAccessToken, 1,
CONVERT(VARBINARY(128), playerSocialNetworkTokenID))) AS
accessTokenRecovered
FROM dbo.playerSocialNetworkTokens;
GO
```

```
CLOSE SYMMETRIC KEY GathelPasswordKey;
GO
```

Actualmente los datos en la base de datos están descifrados.

10) Guía de la demostración

1. Insertar datos de prueba como sa

```
INSERT INTO dbo.players (countryID, email, username, firstName, lastName,
passwordHash, isEmailVerified, isActive, createdAt)
VALUES
```

```
(1, 'ana@demo.com', 'ana', 'Ana', 'Lopez', 'HASH_ANA', 1, 1, SYSDATETIME()),
(1, 'bruno@demo.com', 'bruno', 'Bruno', 'Perez', 'HASH_BRUNO', 1, 1,
SYSDATETIME()),
```

```
(1, 'carla@demo.com', 'carla', 'Carla', 'Soto', 'HASH_CARLA', 1, 1, SYSDATETIME());
```

Después inserta tres proposiciones de ejemplo:

```
INSERT INTO dbo.propositions
(creatorPlayerID, targetPlayerID, propositionStatusID, propositionText,
predictionsDeadline, votingDeadline, createdAt, isActive)
```

```
VALUES
```

```
(1, 2, 1, 'Ana dice que Bruno ganará la maratón', SYSDATETIME(), SYSDATETIME(),
SYSDATETIME(), 1),
```

```
(2, 3, 1, 'Bruno dice que Carla publicará un video', SYSDATETIME(), SYSDATETIME(),
SYSDATETIME(), 1),
```

```
(3, 1, 1, 'Carla dice que Ana aprobará el examen', SYSDATETIME(), SYSDATETIME(),
SYSDATETIME(), 1);
```

Y mete datos sensibles para masking:

```
INSERT INTO dbo.loginAttempts (playerID, attemptedEmail, ipAddress,
wasSuccessful, failureReason, attemptedAt, createdAt)
VALUES
(1, 'ana@demo.com', '10.10.10.10', 0, 'Bad password', SYSDATETIME(),
SYSDATETIME());
```

```
INSERT INTO dbo.paymentAttempts
(paymentMethodID, playerID, operationTypeCodeID, targetEntityType,
targetEntityID, sourceEntityType, sourceEntityID,
amount, currencyID, exchangeRate, exchangeRateID, paymentStatusID, result,
requestPayload, responsePayload,
transactionReference, checksum, postedAt, createdAt)
VALUES
(1, 1, 1, 'proposition', 1, 'wallet', 1,
25.00, 2, 1.000000, 1, 1, 'ok', '{"card":"4111"}', '{"approved":true}',
'TXN-SECRET-001', 'CHK-001', SYSDATETIME(), SYSDATETIME());
```

```
INSERT INTO dbo.playerSocialNetworkTokens
(playerSocialNetworkID, accessTokenHash, refreshTokenHash, tokenExpiresAt,
isActive, createdAt)
VALUES
(1, 'access-token-demo', 'refresh-token-demo', DATEADD(day, 30, SYSDATETIME()),
1, SYSDATETIME());
```

2. Mapear usuarios SQL Server a jugadores para RLS

```
INSERT INTO SecurityLab.UserPlayerMap (dbUserName, playerID)
VALUES
('lab_reader', 1),
('lab_writer', 2),
('lab_auditor', 3);
```

3. Qué debe ejecutar con cada sesión

3.1 lab_reader

```
SELECT * FROM dbo.players;
```

Resultado esperado: fallo por permiso denegado, porque no tiene SELECT directo sobre esa tabla.

```
EXEC dbo.sp_GetPlayerPublicProfile @PlayerID = 1;
```

Resultado esperado: sí funciona y devuelve solo columnas públicas.

```
SELECT propositionID, creatorPlayerID, targetPlayerID, propositionText  
FROM dbo.propositions;
```

Resultado esperado: solo verá las filas que le permita la política RLS

3.2 lab_writer

Ejecuta:

```
INSERT INTO dbo.loginAttempts (playerID, attemptedEmail, ipAddress,  
wasSuccessful, failureReason, attemptedAt, createdAt)  
VALUES (2, 'bruno@demo.com', '192.168.1.50', 1, NULL, SYSDATETIME(),  
SYSDATETIME());
```

Resultado esperado: éxito.

Luego:

```
SELECT * FROM dbo.loginAttempts;
```

Resultado esperado: fallo por DENY SELECT configurado.

3.3 lab_auditor

Si se quiere comprobar antes con la sesión sa se le puede dar acceso al auditor para ver los datos con mask o después para validar

```
GRANT UNMASK TO lab_auditor;  
SELECT email, firstName, lastName  
FROM dbo.players;
```

Resultado esperado: dependiendo del orden se verán los datos con mask o no.

4. Prueba del cifrado con certificado

Como sa, abre la symmetric key.

Consulta la columna cifrada.

Cierra la key.

Explica que el valor guardado está cifrado y que solo se recupera con la jerarquía de claves.

La jerarquía correcta en SQL Server es: Database Master Key → Certificate → Symmetric Key, y luego ENCRYPTBYKEY / DECRYPTBYKEY. DECRYPTBYKEY devuelve NULL si la clave no está abierta.